



Fundamentos da Programação Orientada a Objetos

Heranças entre classes

Olá!

Nesta unidade, aprenderemos sobre **heranças**. Nas empresas, é muito comum ter de lidar com códigos que possuem muitas classes que compartilham comportamentos ou atributos. O conceito de herança ajuda a gerenciar essa complexidade, criando uma **hierarquia de classes**, permitindo reaproveitar código e subdividir a aplicação com mais facilidade.

Vamos lá?

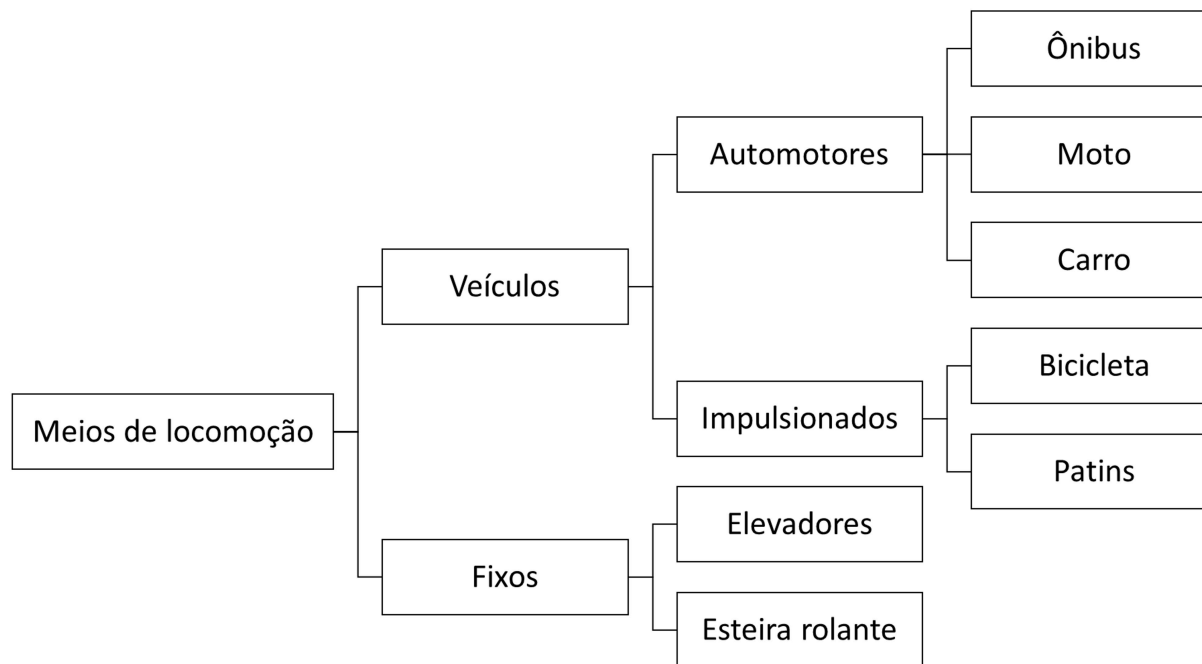


| Hierarquias de classes

Você já aprendeu que objetos se associam entre si. Por exemplo, na composição de um **carro** (todo), precisamos de **partes** como um **motor**, **rodas** e **portas**. Ou, ainda, podemos ter casos de agregação de um objeto: uma rede de fast food pode vender a você um **sanduíche** e um **refrigerante** em um combo, ou ambos separados.

Da mesma forma, quando procuramos entender o mundo à nossa volta, também **agrupamos os objetos por suas características**. Um exemplo clássico está na biologia: quando estudamos os animais, os subdividimos em reinos, filos, raças etc. Assim, sabemos que cães e gatos são bichos completamente diferentes, mas guardam similaridades por serem ambos mamíferos. Embora a biologia tenha essa hierarquia definida formalmente, nós fazemos o tempo todo uma organização mais informal. Um desses exemplos está na figura apresentada a seguir, criada para organizar diferentes tipos de veículos.

Figura 1: Hierarquia dos veículos.



Fonte: Autores (2021)

Observe que:

- Carros, ônibus e motos possuem **características diferentes**: no número de rodas, capacidade de passageiros etc.
- Porém, também possuem várias **características similares**: a impulsão usa um motor, o deslocamento é sobre rodas etc. e isso permite entendê-las como veículos automotores. Isso significa que um carro ou moto possui as mesmas **características e operações** que um veículo automotor, pois eles **são um** tipo mais específico deste tipo de veículo.
- Da mesma forma, os veículos automotores e os impulsionados podem ser agrupados como veículos, uma vez que podem deslocar pessoas livremente de um ponto a outro.

Partindo dessas definições, temos os seguintes termos em POO:

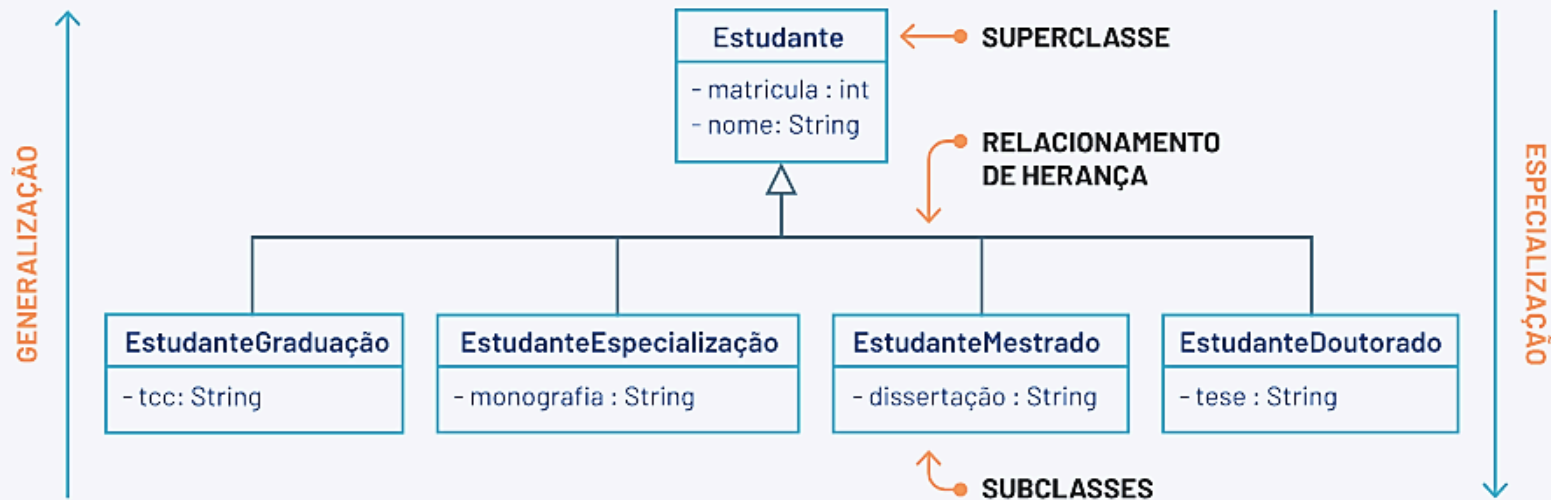
1. **Especialização:** é o **processo** de criar classes a partir de uma classe existente, adicionando atributos e comportamentos específicos. O resultado desse processo são **subclasses** (também chamadas de classes-filhas). Exemplos:
 1. **Ônibus, moto e carro** são subclasses de **automotores**.
 2. **Automotores e impulsioneados** são subclasses de **veículos**.
2. **Generalização:** é o processo inverso da especialização, em que características comuns a várias classes são combinadas em uma **superclasse**(também chamada de classe-pai). É como criar uma versão mais ampla e geral de várias classes. Exemplos:
 1. **Automotores** é uma superclasse de **ônibus, moto e carro**.
 2. **Veículos** é uma superclasse de **automotores e impulsioneados**.

Especialização e generalização

O que acha de pensarmos em outro exemplo de especialização e generalização?

Observe o exemplo de hierarquia apresentado a seguir, representado com UML: as relações de herança são indicadas pelo símbolo do triângulo vazado, que aponta para a classe-pai (ou superclasse).

Figura 2: Hierarquia dos veículos.



Fonte: Autores (2021).

Neste caso, a leitura do diagrama acompanha o sentido da hierarquia:

- Da **subclasse** para a **superclasse** é a generalização: a leitura vai da classe mais específica para a classe mais geral, ou **genérica**.
- Da **superclasse** para a **subclasse** é a especialização: a leitura vai da classe mais genérica para a classe mais **específica**.

Vamos começar com alguns exemplos para demonstrarmos como estas classes interagem entre si.



EXEMPLO

Exemplo de aplicação 1: crie um projeto em Java seguindo as seguintes instruções:

1. Crie três classes: Carro, Moto e Main. Todas as classes fazem parte do pacote *default*.
2. Cole o código para cada arquivo conforme listado a seguir.
3. Execute o projeto a partir da classe Main.

Arquivo Carro.java:

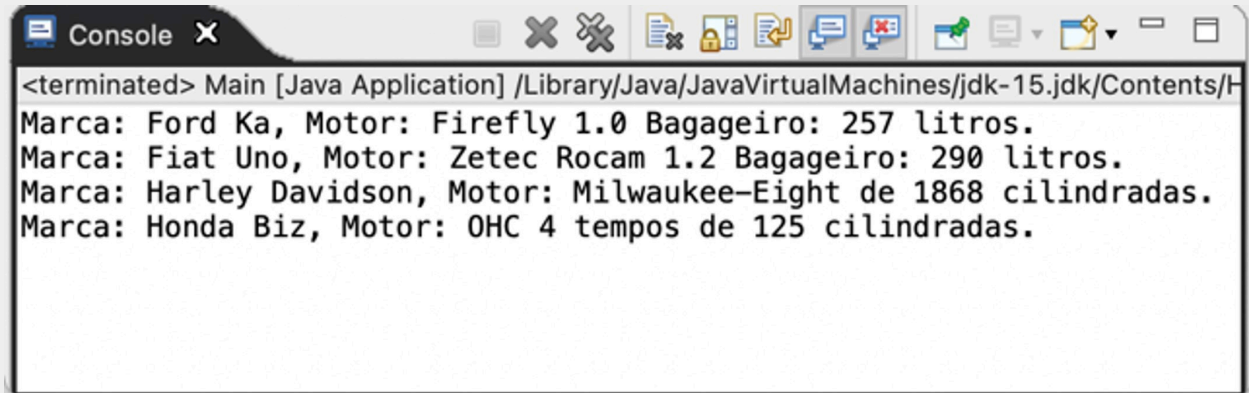
</>

```
1 public class Carro {
2     private String marca;
3     private String motor;
4     private int portaMalas;
5
6     public Carro(String marca, String motor, int portaMalas) {
7         this.marca = marca;
8         this.motor = motor;
9         this.portaMalas = portaMalas;
10    }
11
12    public String getMarca() {
13        return marca;
14    }
15
16    public String getMotor() {
17        return motor;
18    }
19
```

Arquivo Moto.java:

</>

```
1 public class Moto {
2     private String marca;
3     private String motor;
4     private int cilindradas;
5
6     public Moto(String marca, String motor, int cilindradas) {
7         this.marca = marca;
8         this.motor = motor;
9         this.cilindradas = cilindradas;
10    }
11
12    public String getMarca() {
13        return marca;
14    }
15
16    public String getMotor() {
17        return motor;
18    }
19
```

The screenshot shows a Java console window titled "Console" with a standard toolbar. The output text is as follows:

```
<terminated> Main [Java Application] /Library/Java/JavaVirtualMachines/jdk-15.jdk/Contents/H  
Marca: Ford Ka, Motor: Firefly 1.0 Bagageiro: 257 litros.  
Marca: Fiat Uno, Motor: Zetec Rocam 1.2 Bagageiro: 290 litros.  
Marca: Harley Davidson, Motor: Milwaukee-Eight de 1868 cilindradas.  
Marca: Honda Biz, Motor: OHC 4 tempos de 125 cilindradas.
```

Arquivo Main.java:

</>

```
1 import java.util.*;
2
3 public class Main {
4     public static void main(String[] args) {
5         var carros = new ArrayList<Carro>();
6         var motos = new ArrayList<Moto>();
7
8         carros.add(new Carro("Ford Ka", "Zetec Rocam 1.2", 257));
9         carros.add(new Carro("Fiat Uno", "Firefly 1.0", 290));
10
11         motos.add(new Moto("Harley Davidson", "Milwaukee-Eight", 1868));
12         motos.add(new Moto("Honda Biz", "OHC 4 tempos", 125));
13
14         for (var carro : carros) {
15             System.out.println(carro.imprimir());
16         }
17
18         for (var moto : motos) {
19             System.out.println(moto.imprimir());
```

Agora, veja os métodos e atributos nas classes **moto** e **carro**. Consegue perceber algumas similaridades? Temos dois métodos em comum em ambas as classes: **getMarca()** e **getMotor()**. Além disso, o método **imprimir()** é similar nas duas classes. Possuem o mesmo papel: descrever a classe. Porém, sua implementação, difere um pouco.

Agora, veja a classe **main**: são utilizadas duas listas, uma de carros e outra de motos. Ainda, temos dois loops do tipo “**for**” praticamente idênticos e com a mesma função: imprimir os veículos. Veja que, para cada classe incluída, o código seria repetido mais uma vez.

Importante: foi usada a palavra-chave **var** antes das declarações. Você pode utilizá-la em **variáveis locais** para que o Java determine o tipo de dado automaticamente, com base na atribuição. Assim, os comandos:

```
var carros = new ArrayList ();  
var nome = "Orientação a Objetos";
```

São equivalentes à:

```
ArrayList carros = new ArrayList<>();  
String nome = "Orientação a objetos";
```

Porém, apesar de prático, o comando **var** possui **limitações**. Não pode ser usado em atributos e, no caso dele, a inicialização da variável na mesma linha é **obrigatória**.

Bom – se **moto** e **carro** possuem similaridades, poderíamos muito bem generalizar estas duas funções. E será exatamente isto que faremos agora.



EXEMPLO

Exemplo de aplicação 2: crie uma superclasse chamada **veiculo**. Depois, faça os seguintes passos:

1. Copie os métodos e atributos que são idênticos na classe Carro e Moto para a superclasse Veiculo e altere os construtores das classes Carro e Moto para que usem as implementações da superclasse.
2. Altere a declaração das classes Moto e Veiculo para incluir a instrução **extends** Veiculo.
3. Exclua os métodos idênticos das classes Carro e Moto.
4. Crie uma terceira classe Onibus.
 - a. Adicione a ela os atributos do número de passageiros e se é ou não articulado.
 - b. Não esqueça também do método **imprimir()**.

c. Crie alguns ônibus na classe *Main*.

Muitas instruções, não é? Calma – vamos por partes. O primeiro passo será o de criar a superclasse chamada VEICULO. Portanto, crie um arquivo chamado **Veiculo.java**, e coloque o código apresentado a seguir:

</>

```
1 public class Veiculo {
2     private String marca;
3     private String motor;
4
5     public Veiculo(String marca, String motor) {
6         this.marca = marca;
7         this.motor = motor;
8     }
9
10    public String getMarca() {
11        return marca;
12    }
13
14    public String getMotor() {
15        return motor;
16    }
17 }
```

Perceba que este código possui atributos e métodos genéricos de todo veículo. Afinal, todo veículo teria uma marca e um motor. Os métodos **getMarca()** e **getMotor()** funcionariam para todos os veículos.

Agora, o próximo passo será o de conectar **veiculo** com a **moto** e, ainda, **veiculo** com **carro**. Fazemos isso com a palavra-chave **extends**. Veja, a seguir, o código modificado da classe **carro**:

</>

```
1 public class Carro extends Veiculo {
2     private int portaMalas;
3
4     public Carro(String marca, String motor, int portaMalas) {
5         super(marca, motor);
6         this.portaMalas = portaMalas;
7     }
8
9     public int getPortaMalas() {
10         return portaMalas;
11     }
12
13     public String imprimir() {
14         return "Marca: " + getMarca() + ", Motor: " + getMotor() +
15             " Porta malas: " + getPortaMalas() + " litros.";
16     }
17 }
```

Vamos entender o que alteramos aqui? Vejamos:

1. Linha 1: adicionamos o **extends veiculo**. Isso indica que a classe **carro** “estende” (é uma subclasse de) **veiculo**.

a. Isso fará com que todos os métodos (**getMarca()** e **getMotor()**) e atributos (**marca** e **motor**) de **Veiculo** sejam automaticamente importados para a classe **carro**. É por isso que nós apagamos estes métodos e atributos da classe **carro**: eles não são mais necessários já que foram implementados em **veiculo**. Logo, é desnecessário copiar o mesmo código.

2. Linha 5: usamos o **super()** para chamar o construtor da superclasse. Isso é bem útil quando a superclasse tem um construtor com parâmetros que precisam ser inicializados. Por exemplo, **super(marca, motor)** chama o construtor da superclasse **veiculo** que corresponde aos parâmetros fornecidos (isto é: linha 5 do arquivo **Veiculo.java**). A chamada ao construtor da superclasse usando **super()** deve ser a primeira instrução no construtor da subclasse sempre que formos utilizá-la.

O que acha de você tentar por você mesmo fazer as mesmas mudanças na classe **moto**? Depois de tentar, compare o seu resultado com o código a seguir:

</>

```
1  public class Moto extends Veiculo {
2      private int cilindradas;
3
4      public Moto(String marca, String motor, int cilindradas) {
5          super(marca, motor);
6          this.cilindradas = cilindradas;
7      }
8
9      public int getCilindradas() {
10         return cilindradas;
11     }
12
13     public String imprimir() {
14         return "Marca: " + getMarca() + ", Motor: " + getMotor() +
15             " de " + getCilindradas() + " cilindradas.";
16     }
17 }
```

Percebeu que as mudanças que fizemos foram parecidas com o da classe carro, não é? Removemos os métodos que já existiam na superclasse, e incluímos as menções à própria superclasse com o **extends**. Teste o seu código: ele deve funcionar igual à implementação original.

Agora, vamos à implementação da classe **onibus**. Crie o arquivo **Onibus.java** em seu projeto e cole o código a seguir:

```
</>
1  private int passageiros;
2  private boolean articulado;
3
4  public Onibus(String marca, String motor, int passageiros, boolean articulado) {
5      super(marca, motor);
6      this.passageiros = passageiros;
7      this.articulado = articulado;
8  }
9
10 public int getPassageiros() {
11     return passageiros;
12 }
13
14 public boolean isArticulado() {
15     return articulado;
16 }
17
18 public String imprimir() {
19     return "Marca: " + getMarca() + ", Motor: " + getMotor() +
```

Perceba como algumas funcionalidades já vieram “de brinde” graças ao **extends**: informações sobre marca e motor já estão prontas ainda que você não tenha criado uma única linha de código adicional para isso. Interessante, não é?

Agora, para fechar o nosso exercício: modifique o **Main.java** para incluir os novos ônibus conforme o código modificado a seguir – perceba principalmente as mudanças introduzidas nas linhas 7, 15, 16 e 26 a 28:

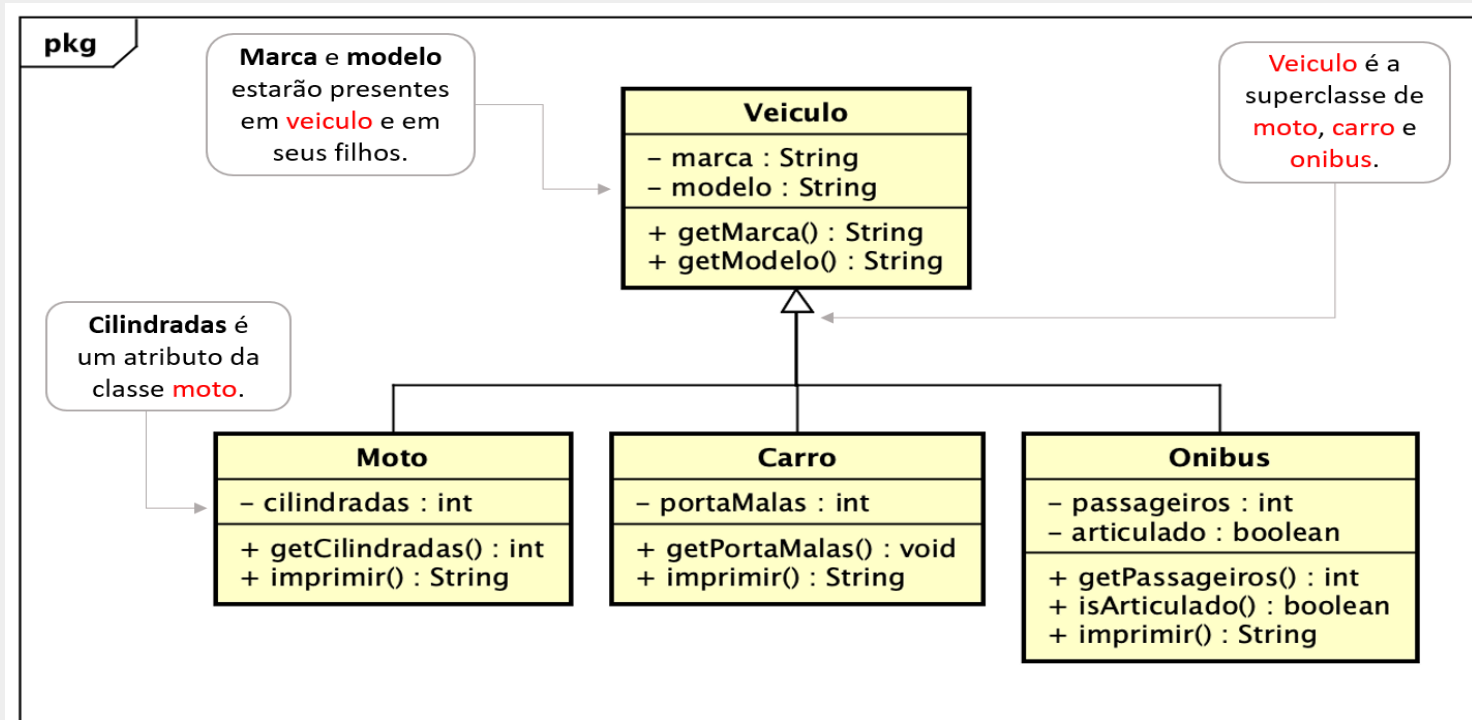
```
</>
```

```
1 import java.util.*;
2
3 public class Main {
4     public static void main(String[] args) {
5         var carros = new ArrayList<Carro>();
6         var motos = new ArrayList<Moto>();
7         var onibus = new ArrayList<Onibus>();
8
9         carros.add(new Carro("Ford Ka", "Zetec Rocam 1.2", 257));
10        carros.add(new Carro("Fiat Uno", "Firefly 1.0", 290));
11
12        motos.add(new Moto("Harley Davidson", "Milwaukee-Eight", 1868));
13        motos.add(new Moto("Honda Biz", "OHC 4 tempos", 125));
14
15        onibus.add(new Onibus("Mercedes Benz", "V8", 50, true));
16        onibus.add(new Onibus("Scania Escolar", "V8", 15, false));
17
18        for (var carro : carros) {
19            System.out.println(carro.imprimir());
20        }
21    }
22 }
```

| Diagrama de classes da herança

Lembra do diagrama de classes UML? Pois bem, também podemos representar as heranças visualmente. Veja o exemplo a seguir:

Figura 4: Diagrama das classes moto, carro, onibus e veiculo.



Fonte: Autores (2021).

Observe o seguinte a partir da especificação apresentada:

1. Todos os atributos são privados (encapsulamento, com modificador -).
2. Todos os métodos são públicos (com modificador +, são a interface das classes).
3. Observe o símbolo do triângulo no conector. A ponta do triângulo indica a classe-pai.
4. Assim como ocorreu no código, não há necessidade de repetir atributos da classe-pai que estejam presentes nas classes-filhas. Isso acontece porque a relação de **herança** é uma relação de tipos. Ou seja, dizemos que o carro **é um tipo** específico de veículo. Portanto, ele obrigatoriamente deve possuir os mesmos atributos e métodos que todos os veículos possuem.

Em Java, cada uma classe pode ter qualquer número de filhos, **mas apenas um único pai**. Outras linguagens, como o C++, permitem mais de um pai (herança múltipla).



DICA

De forma prática, dizemos que a **herança** cria uma relação do gênero “**é um tipo**”, enquanto as **associações** criam relações do gênero “**tem um**”. Se ainda estiver em dúvida, **prefira usar uma associação no lugar de uma herança**.

| Encadeamento de construtores

Todo objeto, ao ser instanciado, precisará ser completamente construído antes de ser utilizado. Por isso, classes-filhas devem chamar um dos construtores da sua classe-pai, por meio da palavra-chave **super**. Isso foi observado no código do exemplo anterior:

</>

```
1  
2  
3    ...  
4    public Carro(String marca, String motor, int portaMalas) {  
5        super(marca, motor);  
6        this.portaMalas = portaMalas;  
7    }
```

Na linha 5, a chamada à **super** fará com que o construtor da classe Veiculo seja acionado. Agora, olhando o código em Veiculo, observe que a marca e o motor foram fornecidos uma vez que são os parâmetros de entrada do construtor.

</>

```
1  
2  
3  
4 ...  
5     public Veiculo(String marca, String motor) {  
6         this.marca = marca;  
7         this.motor = motor;  
8     }
```

É importante que você saiba algumas regras sobre encadeamento de construtores:

1. Construtores não são herdados. Assim, você precisará criar um construtor na classe-filha mesmo que a única coisa que ele faça seja chamar **super**.
2. A chamada ao comando **super** deve estar na primeira linha do construtor.
3. Além do comando **super**, há também o comando **this**, que permite chamar um construtor da mesma classe.
4. Você só pode ter um único comando **super** ou **this** em cada construtor.

Por exemplo, o construtor a seguir poderia ser definido na classe carro para permitir a criação de carros com a capacidade padrão de 135 litros no porta-malas.

```
</>  
1     public Carro(String marca, String motor) {  
2         this(marca, motor, 135); //Chama o construtor com 3 parâmetros de carro  
3     }
```

Ainda confuso? Então, veja o que o código a seguir faria:

```
var gol = new Carro("Gol", "1.0");
```

1. Ele chamaria o construtor `Carro(String marca, String motor)` com os valores “Gol” e “1.0”.
2. A primeira linha desse construtor, como vimos, possui a chamada a `this(marca, motor, 135)`. Isso fará com que o construtor `Carro(String marca, String motor, int portaMalas)` seja chamado, com os valores “Gol”, “1.0” e 135.
3. Na primeira linha deste construtor, encontra-se a chamada à `super(marca, motor)`. Isso faz com que o construtor `Veiculo(String marca, String motor)` seja chamado.
4. Este construtor inicializa os atributos **marca** e **motor** da classe Veiculo com os valores passados (“Gol”, “1.0”).
5. Ao retornar, o construtor de carro inicializa o atributo do `portaMalas` com o valor passado (135).

| Acesso aos membros herdados

Um objeto da classe-filha possuirá todos os **atributos** e **métodos** definidos em sua classe e, ainda, os atributos e métodos da classe-pai. Entretanto, isto não significa necessariamente que a classe-filha poderá acessar todos os atributos e métodos da classe-pai dependendo dos seus modificadores de acesso.

Atributos **private** estão acessíveis somente em sua própria classe (onde foram declarados). Por isso, os atributos Marca e Modelo não serão diretamente acessíveis nas classes-filhas de Veiculo (Carro, Moto e Onibus). Outros modificadores de acesso – seja o **public**, **default** ou **protected** – podem permitir o acesso ao atributo ou método nas classes-filhas. Nesse caso, basta utilizá-los diretamente, prefixando ou não o membro com a palavra-chave **this**, como você faria se ele fosse declarado na própria classe.

Por exemplo, o método **imprimir()** da classe Moto não pode utilizar diretamente os atributos Marca e Modelo, uma vez que foram definidos como **private**. Porém, a classe utilizou os seus **getters**, que são públicos, para acessá-los:

</>

```
1 public class Moto extends Veiculo {  
...  
13     public String imprimir() {  
14         return "Marca: " + getMarca() + ", Motor: " + getMotor() +  
15             " de " + getCilindradas() + " cilindradas.";  
16     }  
17 }
```

Para fixar esse conceito e relembrar os demais modificadores de acesso, que tal utilizarmos o modificador *protected* em um pequeno exercício? Podemos recordar as definições para os modificadores de acesso, como está na unidade 3, encapsulamento, também apresentado no quadro a seguir:

Membros *protected*

Conforme já vimos na unidade 3, um atributo *protected* é **visível e acessível por qualquer classe-filha ou classes no mesmo pacote**. Vamos testar isso na prática.



EXEMPLO

Exemplo de aplicação 3: a partir do exemplo anterior faça as seguintes alterações no código:

Figura 6 – Recordando: modificadores de acesso

Visibilidade				
Modificador em Java	Class	Package	Subclasse*	Mundo
sem modificador (default)	✓	✓	✗	✗
public	✓	✓	✓	✓
protected *	✓	✓	✓	✗
private	✓	✗	✗	✗

Fonte: Autores (2021).

1. Classe **veiculo**: altere a visibilidade do atributo **marca** para *protected* e do atributo **motor** para *default*.
2. Em seguida, altere o método imprimir da classe Moto para utilizar os atributos diretamente.
3. Altere também o método da classe Carro para utilizar os atributos, mas prefixe-os com a palavra-chave **this**.

Portanto, os códigos modificados ficarão conforme os exemplos a seguir. Modificamos as linhas 2 e 3 na classe Veiculo, as linhas 14 e 15 na classe Moto, e as linhas 14 e 15 na classe Carro:

Arquivo **Veiculo.java**:

```
</>
```

```
1 public class Veiculo {
2     protected String marca;
3     String motor;
4
5     public Veiculo(String marca, String motor) {
6         this.marca = marca;
7         this.motor = motor;
8     }
9
10    public String getMarca() {
11        return marca;
12    }
13
14    public String getMotor() {
15        return motor;
16    }
17 }
```

Arquivo **Moto.java**:

</>

```
1 public class Moto extends Veiculo {
2     private int cilindradas;
3
4     public Moto(String marca, String motor, int cilindradas) {
5         super(marca, motor);
6         this.cilindradas = cilindradas;
7     }
8
9     public int getCilindradas() {
10         return cilindradas;
11     }
12
13     public String imprimir() {
14         return "Marca: " + marca + ", Motor: " + motor +
15             " de " + cilindradas + " cilindradas.";
16     }
17 }
```

Arquivo **Carro.java**:

</>


```
1 public class Carro extends Veiculo {
2     private int portaMalas;
3
4     public Carro(String marca, String motor, int portaMalas) {
5         super(marca, motor);
6         this.portaMalas = portaMalas;
7     }
8
9     public int getPortaMalas() {
10         return portaMalas;
11     }
12
13     public String imprimir() {
14         return "Marca: " + this.marca + ", Motor: " + this.motor +
15             " Porta malas: " + this.portaMalas + " litros.";
16     }
17 }
```

Teste o código agora. Você não perceberá nenhuma mudança. Agora, crie um pacote chamado **motos** e mova a classe **moto** para lá.

Atenção: ao mover a classe **Moto** para o pacote **motos**, a sua IDE (como o IntelliJ) pode não atualizar tudo automaticamente. Sempre verifique:

1. Se a primeira linha do arquivo **Moto.java** é **package motos**;
2. Se a classe **Main.java** possui a linha **import motos.Moto**;
3. Se a classe **Veiculo** é **public**, para ser visível fora do seu pacote.

Tente rodar o seu código novamente. É provável que você receba alguns erros na sua classe **moto**:

- 1.java: cannot find symbol: class Veiculo
- 2.java: cannot find symbol: variable marca
- 3.java: cannot find symbol: variable motor

Por que tivemos este erro? Ao mover a classe **moto** para outro **pacote**, o atributo **motor** deixou de ser visível na classe **moto**. Para corrigir, ou teríamos que declará-lo como **protected** (arquivo **Veiculo.java**, linha 3), ou utilizar modificadores de acesso.

Uma das outras formas de eliminar esse problema seria trocar o atributo **motor** para **public**. Mas essa seria uma solução correta? Pensemos: o modificador **public** permite que o atributo sempre seja acessível. Mas essa solução é ruim porque remove o **encapsulamento da classe**. É por isso que usar o **protected** seria mais vantajoso, pois permitiria o acesso direto somente às classes-filhas.

| Sobrescrita de métodos

Vamos nos lembrar do método **imprimir()** que implementamos nas classes carro e moto? Veja só:

</>

```
1 public class Moto extends Veiculo {  
...  
15     public String imprimir() {  
16         return "Marca: " + getMarca() + ", Motor: " + getMotor() +  
17             " de " + getCilindradas() + " cilindradas.";  
18     }  
19 }
```

</>

```
1 public class Carro extends Veiculo {  
...  
13     public String imprimir() {  
14         return "Marca: " + getMarca() + ", Motor: " + getMotor() +  
15             " Porta malas: " + getPortaMalas() + " litros.";  
16     }  
17 }
```

Notou a **similaridade**? Isso ocorre porque a marca e o motor são características de todos os veículos. Por isso, poderíamos imaginar que a classe Veiculo poderia fornecer uma implementação padronizada do método imprimir igual a apresentada a seguir:

</>

```
1 public class Veiculo {  
...  
18     public String imprimir() {  
19         return "Marca: " + getMarca() + ", Motor: " + getMotor();  
20     }  
21 }
```

Isto é, obviamente, uma vantagem. Se incluíssemos agora mais de dez tipos diferentes de veículos em nossa hierarquia, não precisaríamos implementar o método imprimir novamente, do zero. Todos possuiriam o método que, ao ser chamado, imprimiria os atributos marca e o motor.

Mas, se fizéssemos isso, o que aconteceria com esse método nas classes Moto, Carro e Onibus? Eles passam a ser proibidos? Teremos de excluí-los?

A resposta é: **não**. Nós podemos criar versões mais específicas de um método por meio de um recurso chamado de **sobrescrita** (*override*). Se um método de mesma **assinatura** (nome, tipo de retorno e parâmetros) for definido na classe-filha, ele será utilizado daquele ponto em diante na hierarquia.



IMPORTANTE

Sobrescrita (*override*) é um recurso que permite a uma subclasse, ou classe-filha, **redefinir** a implementação de um **método herdado** de uma de suas superclasses ou classes-pai.

Método sobrescrito é um método em uma classe-filha que tem a **mesma assinatura** (**mesmo nome, mesmos parâmetros** e o mesmo **tipo de retorno**) desse mesmo método definido em uma de suas classes-pai.

Vamos praticar um pouco isso.



EXEMPLO

Exemplo de aplicação 4: usando como base a implementação do exemplo 2, adicione o método **imprimir()** à classe **veiculo** usando o código de exemplo que acabamos de mostrar. Em seguida, rode o programa.

Na prática, o seu código em **Veiculo.java** ficará assim:

</>

```
1 public class Veiculo {
2     protected String marca;
3     String motor;
4
5     public Veiculo(String marca, String motor) {
6         this.marca = marca;
7         this.motor = motor;
8     }
9
10    public String getMarca() {
11        return marca;
12    }
13
14    public String getMotor() {
15        return motor;
16    }
17
18    public String imprimir() {
19        return "Marca: " + getMarca() + ", Motor: " + getMotor();
```

Você verá que ele continua executando exatamente igual rodava antes. Agora, exclua o método **imprimir()** da classe-filha **Moto** e teste novamente. O que acontece?

O programa continua rodando, porém um pouco diferente: não veremos mais as cilindradas das motos. Quando invocamos o método **imprimir()** de um objeto da classe-filha que já não tem esse método **imprimir()** próprio, invocaremos o método herdado da classe **veiculo**, apresentando na tela de console apenas os atributos comuns definidos na classe-pai: **marca** e **motor**.

Chamando métodos sobrescritos da classe-pai

Agora que criamos o método imprimir na classe Veiculo, você pode estar se perguntando: haveria alguma forma de utilizá-lo nas classes moto, carro e onibus para evitar a repetição do código, uma vez que o sobrescrevemos nessas classes? Em outras palavras, se Moto, Carro e Onibus também possuem o método **imprimir()**, haveria alguma forma de utilizar a implementação já feita no mesmo método da classe Veiculo?

Sim! E a resposta está na palavra-chave **super**. Podemos usá-la em vez do **this** para indicar que queremos chamar o método da classe-pai. Assim, seria possível agora reaproveitar seu código na hora de implementar o método **imprimir()** da classe Moto, da seguinte forma:

#ParaTodosVerem 

</>

```
1 public class Moto extends Veiculo {  
...  
13     public String imprimir() {  
14         return super.imprimir() + " de " + getCilindradas() + " cilindradas."  
15     }  
16 }
```

Diferentemente do que ocorre nos construtores, você pode utilizar o **super** para invocar métodos do pai em qualquer linha – e até mesmo mais de uma vez. Este reaproveitamento torna os códigos mais sucintos e bem mais fáceis de manter. Embora não seja usual, você poderia até mesmo chamar **super.imprimir()** em outro método, que não o **imprimir()**.



DICA

Toda vez que usamos o **super**, acessamos o método ou atributo da classe-pai. Uma forma de interpretarmos “**super.imprimir()**” é “**método imprimir() da superclasse**”.

Finalmente, lembre-se: todos os métodos da classe-pai são herdados automaticamente. Ou seja, você só precisa utilizar o **super** se a classe-filha sobrescreveu o método e você quer diferenciar a chamada entre a versão da classe-filha e de seu pai, tal como fizemos com Moto e Veiculo.

Anotação `@Override`

Nas primeiras versões do Java, um problema relativamente banal às vezes dava bastante dor de cabeça aos programadores. Digamos que você criasse a classe Trator e, na hora de sobrescrever o método **imprimir()**, confundisse seu nome e digitasse **imprime()**. Entretanto, note que esse engano não geraria qualquer erro. A classe Trator criaria o método **imprime()** como um **novo método**, no momento que você chamasse **imprimir()**, a versão de Veiculo seria chamada.

A partir do Java 5 (isto é: por duas décadas) podemos sinalizar a um método de que nossa intenção era sobrescrevê-lo utilizando a anotação `@Override`:

</>

```
1 | @Override
2 | public String imprimir() {
```

Assim, o compilador poderá emitir um erro caso esse método não exista nas classes-pai.



DICA

Anotações como “`@Override`” ajudam a tornar o nosso código mais legível, pois fica mais claro de que o método é destinado a sobrescrever um método em uma superclasse. Além disso, ajuda a entender a relação entre classes e pode ser uma dica sobre o funcionamento do código.

Sobrescrita: últimos detalhes

Por fim, esteja atento a mais alguns detalhes sobre métodos sobrescritos:

- a. Um método sobrescrito pode aumentar a visibilidade de um método da classe-pai, mas nunca a diminuir.
- b. Um método **private** é exclusivo da classe em que foi declarado e, portanto, não pode ser sobrescrito.
- c. Para impedir um método de ser sobrescrito de um ponto em diante da hierarquia de classes, basta declará-lo como **final**, por exemplo:
`public final String imprimir()`.
- d. Um método sobrescrito pode retornar um tipo mais específico do tipo retorno do mesmo método da classe-pai. Isso é chamado de **tipo de retorno covariante**. Por exemplo: se a classe `veiculo` possuir um método chamado `getPneu()` que retorna um objeto do tipo `pneu`, a classe `moto` poderia ter o método `getPneu()` retornando o tipo `PneuDeMoto`, filho (subclasse) de `pneu`, e isso ainda seria uma sobrescrita.

| Sobrecarga de métodos

Um conceito similar, mas não relacionado à herança, é o de **sobrecarga** (*overload*) de método. Ela ocorre quando criamos um método com mesmo nome, mas parâmetros diferentes. Isso fará com que o Java escolha qual versão do método deve ser usada de acordo com os parâmetros fornecidos.



DICA

Sobrecarga (overload) é um recurso que permite que **métodos diferentes**, na mesma classe, tenham o **mesmo nome**, mas **assinaturas diferentes** (parâmetros ou tipos de retorno diferentes).



IMPORTANTE

Não confunda a sobrescrita (*override*) com a sobrecarga (*overload*). Lembre-se: na sobrescrita, criamos um método em classe herdeira com a mesma assinatura do método na classe-pai (mesmo nome, parâmetros e tipo de retorno). Na sobrecarga, criamos um método, na mesma classe, com mesmo nome, mas parâmetros diferentes, podendo também ter tipo de retorno diferente.

Utilizando a sobrecarga

Vamos testar a sobrecarga com um exemplo?



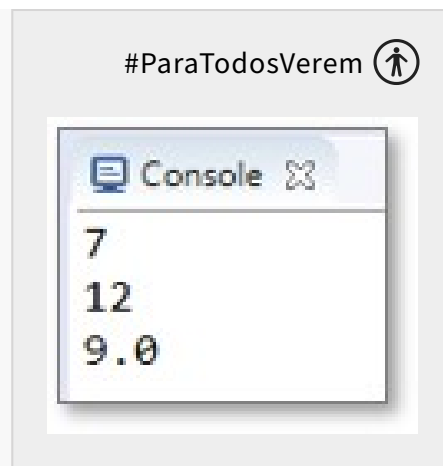
EXEMPLO

Exemplo de aplicação 5: crie um projeto em Java contendo uma única classe com o nome de **soma**. Esta classe deve possuir três métodos, todos chamados de **somar()**. Um dos métodos terá como parâmetros dois números inteiros. Outro, dois números do tipo *double*. Outro, três números inteiros.

Veja como seria um exemplo de implementação:

```
</>
```

```
1 public class Soma {
2     // Overloaded (sobrecarga) de somar(). Esta soma tem 2 parâmetros int;
3     // retorno int
4     public int somar(int x, int y)
5     {
6         return (x + y);
7     }
8
9     // Overloaded (sobrecarga) de somar(). Esta soma tem 3 parâmetros int;
10    // retorno int
11    public int somar(int x, int y, int z)
12    {
13        return (x + y + z);
14    }
15
16    // Overloaded (sobrecarga) de somar(). Esta soma tem 2 parâmetros double;
17    // retorno double
18    public double somar(double x, double y)
19    {
```



Observe a sobrecarga (overload) do método **somar()**, na classe **soma** existem três implementações diferentes (linhas 4, 11 e 18). Também veja que chamamos o método **somar()** nas linhas 25, 26 e 27. Agora, como o compilador sabe qual dessas três implementações usar para cada uma das chamadas? O **compilador** decide qual método **somar()** será invocado dependendo da forma com que são chamados.

Veja essas diferenças nas invocações no método **main()**: a linha 25 usa como parâmetros dois números inteiros. Logo, ela deverá invocar o método da linha 4. Já a linha 26 usa três números inteiros. A única implementação que usa três parâmetros assim está na linha 11. Finalmente, a linha 27 usa dois números decimais. O compilador sabe que a única implementação que aceita dois números decimais é a da linha 18.

No final desta unidade, teremos alguns exercícios para praticar sobrecarga de métodos. Dito isso, gostaria de te dar algumas dicas para usar sobrecarga corretamente:

1. **Diferentes listas de parâmetros:** para sobrecarregar um método, você **precisa alterar a lista de parâmetros**. Você pode alterar a quantidade, o tipo ou a ordem dos parâmetros.
2. **Não confie na alteração do tipo de retorno:** apenas mudar o tipo de retorno do método não é suficiente para sobrecarregar um método. Se você tiver dois métodos com o mesmo nome e a mesma lista de parâmetros, mas com tipos de retorno diferentes, isso resultará em um erro de compilação.
3. **Use para fazer ações parecidas:** a sobrecarga de métodos é mais útil quando você deseja realizar ações semelhantes que requerem diferentes tipos ou números de parâmetros. Por exemplo, você pode ter um método para adicionar números onde um método aceita dois inteiros, outro aceita três inteiros, e assim por diante.

4. **Boas práticas de nomenclatura:** mesmo que o Java permita a sobrecarga de métodos, usar nomes de métodos que descrevam claramente o que o método faz pode tornar seu código mais legível. Se os métodos com o mesmo nome tiverem comportamentos muito diferentes pode talvez valer mais a pena usar nomes diferentes para os métodos.

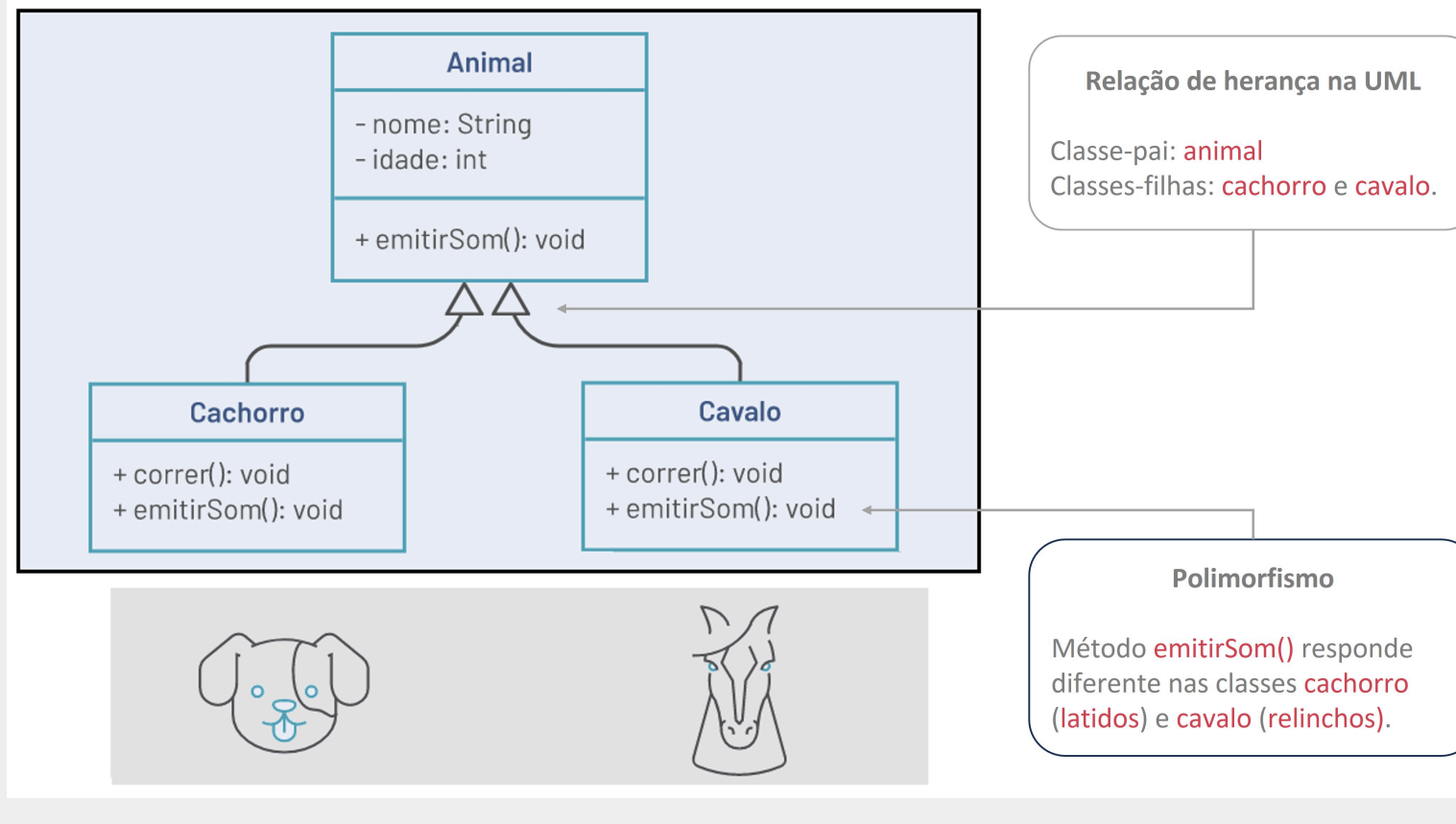
| Polimorfismo

Muitas vezes, entendemos que uma **mesma operação** está presente em **objetos diferentes**, e por isso ela pode ser realizada de **maneiras diferentes**. Isto é: o seu significado (semântica) não muda, embora talvez sua **forma** seja muito diferente. Na da orientação a objetos, esse processo é conhecido como **polimorfismo**, que é possível em relacionamentos de herança entre classes.

Herança: mecanismo que permite que uma subclasse **estenda** uma classe-pai, ou superclasse, provendo **reuso de código** (atributos e métodos), que pode ser incrementado e/ou alterado.

Polimorfismo (**polis** = muitas, **morphos** = formas): na POO, o polimorfismo permite enviar uma mesma mensagem (invocação de método) para **diferentes objetos** de classes herdeiras da mesma superclasse (herança), e fazê-los **responder de maneira diferente**, ou seja, da forma mais apropriada para cada objeto.

Vamos pensar em um exemplo? Analise a imagem a seguir:



Fonte: Autores (2021).

Já conseguimos perceber alguns conceitos a partir do que vimos até o momento:

- Animal: **superclasse** ou **classe-pai**.
- Cachorro e cavalo: **subclasses** ou **classes-filhas**.

Logo: cachorro é **um tipo** de animal, e cavalo é **um tipo** de animal.

Agora, veja também a sobrescrita (override) no método **emitirSom()**: ele é **genérico** na classe **animal**, e é **especializado** nas classes **cachorro** e **cavalo**.

+ Herança

- Um objeto de **animal** é genérico, e seu comportamento **emitirSom()** apenas faz barulhos.
- Já **cachorro** é um **animal** especializado, e seu comportamento é **emitirSom()** de latidos.
- Da mesma maneira, **cavalo** também é um **animal** especializado, e seu comportamento é **emitirSom()** de relinchos.

+ Polimorfismo

- Ele existe na herança quando um objeto pode se **comportar de maneiras diferente** ao receber uma mensagem, ou seja, quando um método seu é invocado.
- Ex.: **emitirSom()** é um método comum a todos os **animais**, mas que responde diferente, dependendo do tipo do **animal**: **cachorro** (latidos) e **cavalo** (relinchos).

Vamos continuar com o tema **polimorfismo** com as classes de veículo, moto e carro: neste caso, temos mais um exemplo que pode ser observado no método **imprimir()**. Obviamente, os dados impressos por uma moto ou carro mudam, mas não muda o fato de que a operação em si, de listar esses dados, é a mesma.

Isso ocorre em várias situações, por exemplo, você pode **abrir** uma caixa, um pirulito e a sua boca. A ação em si é a mesma: **abrir**. Mas a forma que você abre as três coisas é radicalmente diferente, não concorda?

Quando falamos em herança, lembre-se que dissemos que ela cria uma relação de tipos. Ou seja, quando dizemos que Moto é uma classe-filha de Veiculo, estamos também dizendo que a uma Moto **é um tipo** Veiculo. Isso também significa dizer que uma variável do tipo Veiculo poderá apontar para objetos de qualquer classe-filha, como no exemplo a seguir:

```
Veiculo v = new Moto("Harley Davidson", "Milwaukee-Eight", 1868);
```

Essa linha pode ser lida como: “**o veículo v é uma moto**”. O Java só permitirá que chamemos em **v** os métodos que seriam permitidos à classe veiculo, não os específicos da classe moto. Isso inclui até mesmo o método imprimir:

```
v.imprimir();
```

Agora, qual versão do método seria chamada? A da classe Moto, ou a da classe Veiculo? Como o tipo da variável é Veiculo, você poderia imaginar que a versão geral do método será chamada. Mas não é isso que ocorre. Pense um pouco: Se te dissermos “o animal A é um gato. Que som faz A?”. Você obviamente responderá “um miado”. E, com o Java, não é diferente.

Esta característica, de o método imprimir de veículo “mudar” seu comportamento de acordo com o tipo específico do objeto sendo referenciado, é chamada de **polimorfismo** (**polis** = muitas, **morphos** = formas).



DICA

Graças ao **polimorfismo**, o Java sempre considerará o tipo **mais específico** da **instância** (objeto), independentemente de qual seja o tipo da variável que faz referência a ele.

Isso não só torna a sobrescrita muito mais poderosa, como simplifica drasticamente trechos de nosso código. Podemos imaginar que áreas do código tratam de **camadas de abstração** diferentes. Ficou confuso? Então, que tal relermos o método *main*?

```
</>
1  import java.util.*;
2
3  public class Main {
4      public static void main(String[] args) {
5          var carros = new ArrayList<Carro>();
6          var motos = new ArrayList<Moto>();
7          var onibus = new ArrayList<Onibus>();
8
9          carros.add(new Carro("Ford Ka", "Zetec Rocam 1.2", 257));
10         carros.add(new Carro("Fiat Uno", "Firefly 1.0", 290));
11
12         motos.add(new Moto("Harley Davidson", "Milwaukee-Eight", 1868));
13         motos.add(new Moto("Honda Biz", "OHC 4 tempos", 125));
14
15         onibus.add(new Onibus("Mercedes Benz", "V8", 50, true));
16         onibus.add(new Onibus("Scania Escolar", "V8", 15, false));
17
18         for (var carro : carros) {
19             System.out.println(carro.imprimir());

```

Por que há três repetições de código? Três listas? Três fors? Se prestamos atenção, o método *main* está criando um conjunto de veículos e imprimindo suas características. Se incluíssemos outra classe (trator, por exemplo), provavelmente teríamos que criar outra lista e outro for, para realizar **a mesma tarefa**.

Mas o método *main* quer lidar simplesmente com objetos do veículo, e não um veículo específico. Ou seja, ele está em uma camada de abstração mais alta, pois foi feito para lidar com uma classe acima da hierarquia. E, para ele, pouco importa qual veículo específico seja esse ou como ele imprima seus dados.



EXEMPLO

Exemplo de aplicação 6: modifique a classe *main* para que todos os veículos estejam em uma única lista.

Lembre-se: Moto, Carro e Onibus **são** veículos. Por isso, nada nos impediria de os adicionarmos a um *List<Veiculo>*. E lembre-se também que o Java saberá qual versão do método **imprimir()** ele deve chamar, graças ao polimorfismo. Assim, veja como poderíamos reescrever esse código:

</>

```

1  import java.util.*;
2
3  public class Main {
4      public static void main(String[] args) {
5          var veiculos = new ArrayList<Veiculo>();
6
7          veiculos.add(new Carro("Ford Ka", "Firefly 1.0", 257));
8          veiculos.add(new Carro("Fiat Uno", "Zetec Rocam 1.2", 290));
9          veiculos.add(new Moto("Harley Davidson", "Milwaukee-Eight", 1868));
10         veiculos.add(new Moto("Honda Biz", "OHC 4 tempos", 125));
11         veiculos.add(new Onibus("Mercedes Benz", "V8", 50, true));
12         veiculos.add(new Onibus("Scania Escolar", "V8", 15, false));
13
14         for (var veiculo : veiculos) {
15             System.out.println(veiculo.imprimir());
16         }
17     }
18 }

```

Viu como ficou mais simples? Isso porque agora o método *main* pode tratar todos os veículos de maneira uniforme!

Conversões de tipos

Como vimos, uma referência pode conter tipos de sua classe-filha sem qualquer problema. Esse comando sempre será válido:

```
Veiculo v = new Carro(); //O carro é um Veículo
```

Entretanto, a operação contrária não é permitida. Afinal, qualquer carro certamente será um veículo, mas nem todo veículo será um carro (ele pode ser uma moto, um ônibus, ou qualquer outro tipo de veículo).

Em Java, podemos testar o tipo do objeto apontado por uma referência por meio do operador *instanceof*. Ele retornará verdadeiro caso a variável indicada do lado esquerdo seja do tipo da classe indicado do lado direito, ou sua filha, direta ou indiretamente (ou seja, se é a mesma classe, ou uma classe acima na hierarquia, independentemente de sua altura).

A partir do momento em que temos essa certeza, podemos utilizar a operação *type casting* para converter a referência em um tipo inferior da hierarquia. Por exemplo:

```
//A variável v é um Carro?  
if (v instanceof Carro) {  
    //Em caso positivo, convertemos  
    Carro carro = (Carro)v; //Down casting - cast "para baixo"  
    //E agora podemos utiliza-la  
    System.out.println("Capacidade do porta malas:" +  
        carro.getPortaMalas());  
}
```



DICA

type casting é o termo dado para converter uma variável de um tipo para outro. Isso inclui também converter números para *strings*, *strings* para números. Em Raciocínio Computacional você deve ter feito algo como `int(input("Digite um número"))` em Python, não? Aquele `int()` convertia um texto para número. Em outras palavras, você já esteve trabalhando com *type casting* desde o início do curso.

O type casting também é necessário quando você converte um número de ponto flutuante (*float*, *double*) em um número inteiro. Ele é o operador para conversões perigosas (no caso dos números, pode ser que haja casas decimais que seriam descartadas).

E o que ocorre se a conversão do objeto `v` não for possível? O Java disparará um erro do tipo *ClassCastException*, e você deverá modificar o seu código para que a conversão seja válida.

Por fim, é importante ressaltar que esse tipo a operação de *instanceof* é relativamente raro. Normalmente, o polimorfismo e a hierarquia de tipos tornarão esse tipo de conversão desnecessária, resolvendo problemas com muita elegância. Desconfie do seu código caso você esteja fazendo esse tipo de testes o tempo todo.

Vamos para mais um exemplo de polimorfismo e sobrescrita?



EXEMPLO

Exemplo de aplicação 7: tente ler o código a seguir sem testá-lo em sua IDE. O que você acha que aparecerá na tela para o usuário?

</>

```
1 class Foo {
2     public void a() {
3         System.out.println("a");
4     }
5
6     public void b() {
7         System.out.println("b");
8     }
9
10    void print() {
11        a();
12        b();
13    }
14 }
15
16 class Bar extends Foo {
17     public void a() {
18         System.out.println("A");
19     }
```

O tipo da variável de referência é **foo**. Porém, ela aponta para um objeto **bar**. Graças ao polimorfismo, devemos levar em consideração esse objeto ao avaliar os métodos.

Na linha 23, **print()** é chamado. O objeto **bar** herda o método **print()** da classe **Foo** sem alterações, e por isso executa a lógica definida no pai (isto é, **Foo**).

Na linha 11, o método **print()** chamará o método **a()**. Como o objeto é do tipo **bar**, e este possui uma sobrescrita do método (linha 17), sua versão será chamada imprimindo “A”.

Em seguida, na linha 12, o método **b()** será chamado. Não há sobrescrita, por isso, o valor “b” será impresso.

Por isso, o resultado que será mostrado na tela é a string “Ab”.



DICA

Exercícios para analisar o resultado de um código sem executá-lo (como o exemplo apresentado) são comuns em algumas empresas.

Exercícios de fixação



EXERCÍCIO

1. Suponha que você esteja projetando o sistema de back-end para um aplicativo de comércio eletrônico como a Amazon. Crie uma classe genérica chamada **produto** que contenha informações que todos os produtos têm em comum (como **idProduto**, **nome**, **preco**). Agora, crie classes especializadas como **livro**, **eletronico** e **roupa**, cada uma com atributos adicionais relevantes para esse tipo de produto.
2. Crie uma classe **ContaBancaria** com atributos privados como **saldo** e **numeroDaConta**. Crie uma classe **ContaCorrente** que herda de **ContaBancaria** e adiciona um atributo **limiteDeCredito**. Certifique-se de usar **protected** ou **private** corretamente para encapsular os atributos.
3. Vamos pensar no Airbnb. Crie uma classe-base **acomodacao** que possui um método **precoPorNoite()**. Agora crie subclasses **casa** e **apartamento** que sobrescrevem esse método para fornecer preços diferentes por noite.
4. Para o Uber, crie uma classe **viagem** com um método **calcularPreco()**. Sobrecarregue esse método para aceitar diferentes conjuntos de parâmetros. Por exemplo, um método que aceita apenas a **distância** da viagem, outro que aceita **distância** e **horário** (porque o preço pode variar dependendo da hora do dia).
5. Você está no processo seletivo de estágios do Spotify. Para isso, crie uma classe **midia** com um método **play()**. Agora crie classes **musica** e **podcast** que herdam de **midia** e implementam o método **play()** de maneiras diferentes. Em

uma classe **playlist**, você pode ter um ArrayList de **midia**, permitindo que você chame o método **play()** em qualquer item da playlist, seja ele uma música ou podcast.

6. Você está no processo seletivo d'O Boticário. Nele, você recebeu o seguinte problema: você tem uma classe **produto** e subclasses como **perfume** e **maquiagem**. Em uma classe **CarrinhoDeCompras**, você tem uma lista de **produto**. Em um determinado momento, você quer tratar um **produto** em seu carrinho como **perfume** para acessar métodos específicos dessa subclasse. Demonstre como você pode usar typecasting para fazer isso de maneira segura.



Resolução do exercício



Exercício 1

</>

```
1 class Produto {
2     private String idProduto;
3     private String nome;
4     private double preco;
5
6     public Produto(String idProduto, String nome, double preco) {
7         this.idProduto = idProduto;
8         this.nome = nome;
9         this.preco = preco;
10    }
11
12    public String getIdProduto() {
13        return idProduto;
14    }
15
16    public String getNome() {
17        return nome;
18    }
19 }
```

Exercício 2

</>

```
1 class ContaBancaria {
2     protected double saldo;
3     private String numeroDaConta;
4
5     public ContaBancaria(String numeroDaConta, double saldoInicial) {
6         this.numeroDaConta = numeroDaConta;
7         this.saldo = saldoInicial;
8     }
9
10    public String getNumeroDaConta() {
11        return this.numeroDaConta;
12    }
13
14    public double getSaldo() {
15        return this.saldo;
16    }
17
18    public void depositar(double valor) {
19        if (valor > 0) {
```

Exercício 3

</>

```
1 class Acomodacao {
2     public double precoPorNoite() {
3         return 100.0; // Preço padrão por noite.
4     }
5 }
6
7 class Casa extends Acomodacao {
8     @Override
9     public double precoPorNoite() {
10         return 150.0; // Preço por noite para Casa.
11     }
12 }
13
14 class Apartamento extends Acomodacao {
15     @Override
16     public double precoPorNoite() {
17         return super.precoPorNoite() * 1.2; // Preço por noite para Apartamento.
18     }
19 }
```

Exercício 4

</>

```
1 class Viagem {
2     // Preço base da viagem
3     private static final double PRECO_BASE = 2.0;
4     // Preço por quilômetro
5     private static final double PRECO_POR_KM = 1.0;
6     // Preço por quilômetro no horário de pico
7     private static final double PRECO_PICO_POR_KM = 2.0;
8
9     // Método para calcular o preço apenas com a distância
10    public double calcularPreco(double distancia) {
11        return PRECO_BASE + PRECO_POR_KM * distancia;
12    }
13
14    // Método sobrecarregado para calcular o preço com distância e horário
15    public double calcularPreco(double distancia, boolean horarioDePico) {
16        if (horarioDePico) {
17            return PRECO_BASE + PRECO_PICO_POR_KM * distancia;
18        } else {
19            return calcularPreco(distancia);
20        }
21    }
22 }
```

Exercício 5

</>

```
1 import java.util.ArrayList;
2
3 class Midia {
4     public void play() {
5         // Método para ser sobrescrito pelas subclasses
6     }
7 }
8
9 class Musica extends Midia {
10     @Override
11     public void play() {
12         System.out.println("Tocando música...");
13     }
14 }
15
16 class Podcast extends Midia {
17     @Override
18     public void play() {
19         System.out.println("Tocando podcast...");
```

Exercício 6

</>

```
1 import java.util.ArrayList;
2
3 class Produto {
4     private String nome;
5
6     public Produto(String nome) {
7         this.nome = nome;
8     }
9
10    public String getNome() {
11        return nome;
12    }
13 }
14
15 class Perfume extends Produto {
16     private String fragancia;
17
18     public Perfume(String nome, String fragancia) {
19         super(nome);
```

| Referências Bibliográficas

GODOY, V. **Programação orientada a objetos I**. Curitiba: IESDE, 2019.

HORSTMANN, C. S.; CORNELL, G. **Core Java – Volume I**. 8. ed. São Paulo: Pearson, 2010.

